비잔틴 장애 처리

개요

비잔틴 장애 허용성(Byzantine Fault Tolerance)은 CometBFT의 핵심 특성 중 하나입니다. 이 문서는 CometBFT가 악의적이거나 오작동하는 밸리데이터들을 어떻게 감지하고 처리하는지 상세히 설명합니다. 비잔틴 장애는 네트워크의 최대 1/3 미만의 노드가 임의적으로 행동하는 상황을 의미하며, 이러한 상황에서도 시스템의 안전성과 활성을 보장해야 합니다.

비잔틴 장애의 유형

1. 중복 투표 (Duplicate Vote)

가장 일반적인 비잔틴 장애로, 한 밸리데이터가 같은 높이와 라운드에서 서로 다른 블록에 대해 투표하는 경우입니다.

```
// 중복 투표 예시

voteA := &types.Vote{

Height: 100,

Round: 0,

Type: types.PrevoteType,

BlockID: blockID1, // 블록 A에 투표

Signature: signatureA,

}
```

```
voteB := &types.Vote{

Height: 100,

Round: 0,

Type: types.PrevoteType,

BlockID: blockID2, // 블록 B에 투표 (다른 블록!)

Signature: signatureB,

}
```

2. 라이트 클라이언트 공격 (Light Client Attack)

라이트 클라이언트를 속이기 위해 잘못된 블록 헤더를 제공하는 공격입니다. 세 가지 유형이 있습니다:

2.1 루나틱 공격 (Lunatic Attack)

- 결정론적 필드가 잘못된 블록 헤더 제공
- ValidatorsHash, AppHash 등이 올바른 상태 전이와 다름

2.2 등가 공격 (Equivocation Attack)

- 같은 라운드에서 두 개의 다른 블록에 커밋
- 두 블록 모두 유효한 상태 전이를 나타냄

2.3 기억상실 공격 (Amnesia Attack)

- 이전에 잠긴 블록을 무시하고 다른 블록에 투표
- 잠금 메커니즘을 위반하는 행위

장애 감지 메커니즘

1. 합의 과정에서의 실시간 감지

```
func (cs *State) tryAddVote(vote *types.Vote, peerID p2p.ID) (bool, error) {
// 기존 투표 확인
existingVote := cs.Votes.GetVote(vote.ValidatorIndex, vote.Type, vote.Round)
if  existingVote != nil {
// 중복 투표 감지
if !existingVote.BlockID.Equals(vote.BlockID) {
// 서로 다른 블록에 대한 투표 발견
cs.evpool.ReportConflictingVotes(existingVote, vote)
cs.Logger.Info("Detected conflicting votes",
"validator", vote.ValidatorAddress,
"height", vote.Height,
"round", vote.Round,
"vote_a", existingVote.BlockID,
"vote_b", vote.BlockID,
)
return false, ErrConflictingVotes{
VoteA: existingVote,
VoteB: vote,
}
}
// 동일한 투표의 중복 수신
```

```
cs.metrics.DuplicateVote.Add(1)
return false, nil
}
return  cs.addVote(vote, peerID)
}
```

2. 증거 풀에서의 처리

```
func (evpool *Pool) ReportConflictingVotes(voteA, voteB *types.Vote) {
evpool.mtx.Lock()
defer  evpool.mtx.Unlock()
// 합의 버퍼에 충돌하는 투표 저장
// 블록이 커밋된 후 증거로 변환됨
evpool.consensusBuffer = append(evpool.consensusBuffer, duplicateVoteSet{
VoteA: voteA,
VoteB: voteB,
})
}
```

3. 증거 생성 과정

```
func (evpool *Pool) processConsensusBuffer(state sm.State) {
for  _, voteSet := range  evpool.consensusBuffer {
```

```go
// 투표 높이에 따른 밸리데이터 세트 로드
var valSet *types.ValidatorSet
var blockTime time.Time
if voteSet.VoteA.Height == state.LastBlockHeight {
valSet = state.LastValidators
blockTime = state.LastBlockTime
} else {
// 과거 높이의 밸리데이터 세트 로드
valSet, err = evpool.stateDB.LoadValidators(voteSet.VoteA.Height)
if err != nil {
continue
}
blockMeta := evpool.blockStore.LoadBlockMeta(voteSet.VoteA.Height)
blockTime = blockMeta.Header.Time
}
// 중복 투표 증거 생성
dve, err := types.NewDuplicateVoteEvidence(
voteSet.VoteA,
voteSet.VoteB,
blockTime,
valSet,
)
if err != nil {
```

```go
continue
}
// 증거 풀에 추가
if err := evpool.addPendingEvidence(dve); err == nil {
evpool.evidenceList.PushBack(dve)
}
}
// 버퍼 초기화
evpool.consensusBuffer = make([]duplicateVoteSet, 0)
}
```

증거 검증 로직

1. 중복 투표 증거 검증

```go
func (dve *DuplicateVoteEvidence) ValidateBasic() error {
if dve.VoteA == nil || dve.VoteB == nil {
return fmt.Errorf("one or both votes are nil")
}
// 기본 투표 검증
if err := dve.VoteA.ValidateBasic(); err != nil {
return fmt.Errorf("invalid VoteA: %w", err)
}
```

```go
if err := dve.VoteB.ValidateBasic(); err != nil {
return fmt.Errorf("invalid VoteB: %w", err)
}
// 같은 밸리데이터의 투표인지 확인
if !dve.VoteA.ValidatorAddress.Equals(dve.VoteB.ValidatorAddress) {
return fmt.Errorf("votes are from different validators")
}
// 같은 높이, 라운드, 타입인지 확인
if dve.VoteA.Height != dve.VoteB.Height ||
dve.VoteA.Round != dve.VoteB.Round ||
dve.VoteA.Type != dve.VoteB.Type {
return fmt.Errorf("votes are not conflicting")
}
// 서로 다른 블록에 대한 투표인지 확인
if dve.VoteA.BlockID.Equals(dve.VoteB.BlockID) {
return fmt.Errorf("votes are for the same block")
}
// 투표가 사전순으로 정렬되어 있는지 확인
if strings.Compare(dve.VoteA.BlockID.Key(), dve.VoteB.BlockID.Key()) >= 0 {
return errors.New("duplicate votes in invalid order")
}
return nil
```

```
}
```

2. 증거 검증 파이프라인

```
func (evpool *Pool) verify(evidence types.Evidence) error {
switch ev := evidence.(type) {
case *types.DuplicateVoteEvidence:
return evpool.verifyDuplicateVote(ev)
case *types.LightClientAttackEvidence:
return evpool.verifyLightClientAttack(ev)
default:
return fmt.Errorf("unrecognized evidence type: %T", evidence)
}
}

func (evpool *Pool) verifyDuplicateVote(dve *types.DuplicateVoteEvidence) error
// 1. 기본 검증
if err := dve.ValidateBasic(); err != nil {
return err
}
// 2. 만료 확인
if evpool.isExpired(dve.Height(), dve.Time()) {
return fmt.Errorf("evidence is expired")
```

```
}
// 3. 밸리데이터 세트 로드
valSet, err := evpool.stateDB.LoadValidators(dve.Height())
if err != nil {
return err
}
// 4. 밸리데이터 존재 확인
_, validator := valSet.GetByAddress(dve.VoteA.ValidatorAddress)
if validator == nil {
return fmt.Errorf("validator not in validator set")
}
// 5. 서명 검증
chainID := evpool.state.ChainID
if !validator.PubKey.VerifySignature(
dve.VoteA.SignBytes(chainID), dve.VoteA.Signature) {
return fmt.Errorf("invalid signature on VoteA")
}
if !validator.PubKey.VerifySignature(
dve.VoteB.SignBytes(chainID), dve.VoteB.Signature) {
return fmt.Errorf("invalid signature on VoteB")
}
return nil
```

```
}
```

증거 전파 및 커밋

1. 증거 전파

```
// 블록 제안 시 증거 포함
func (cs *State) createProposalBlock() (*types.Block, error) {
// 증거 풀에서 증거 수집
evidence, _ := cs.evpool.PendingEvidence(cs.state.ConsensusParams.Evidence.MaxBy
// 블록에 증거 포함
block := &types.Block{
Header: header,
Data: types.Data{
Txs: txs,
},
Evidence: types.EvidenceData{
Evidence: evidence,
},
LastCommit: lastCommit,
}
return  block, nil
```

```
}
```

2. 증거 검증 (블록 수신 시)

```
func (evpool *Pool) CheckEvidence(evList types.EvidenceList) error {
hashes := make([][]byte, len(evList))
for  idx, ev := range  evList {
// 이미 커밋된 증거인지 확인
if  evpool.isCommitted(ev) {
return &types.ErrInvalidEvidence{
Evidence: ev,
Reason: ErrEvidenceAlreadyCommitted,
}
}
// 증거 검증
if  err := evpool.verify(ev);  err != nil {
return &types.ErrInvalidEvidence{
Evidence: ev,
Reason: err,
}
}
// 중복 증거 확인
hashes[idx] = ev.Hash()
```

```go
for i := idx - 1; i >= 0; i--
if bytes.Equal(hashes[i], hashes[idx])
return &types.ErrInvalidEvidence{
Evidence: ev,
Reason: ErrDuplicateEvidence,
}
}
}
// 증거 풀에 추가
if err := evpool.addPendingEvidence(ev); err != nil
evpool.logger.Error("Failed to add evidence", "err", err)
}
}
return nil
}
```

장애 복구 메커니즘

1. 타임아웃 기반 복구

```go
func (cs *State) handleTimeout(ti timeoutInfo, rs cstypes.RoundState)
// 현재 상태와 타임아웃 정보 비교
if ti.Height != rs.Height || ti.Round < rs.Round
```

```
return  // 오래된 타임아웃 무시
}
switch  ti.Step {
case  cstypes.RoundStepPropose:
// 제안 타임아웃 - 사전 투표로 진행
cs.enterPrevote(ti.Height, ti.Round)
case  cstypes.RoundStepPrevoteWait:
// 사전 투표 타임아웃 - 사전 커밋으로 진행
cs.enterPrecommit(ti.Height, ti.Round)
case  cstypes.RoundStepPrecommitWait:
// 사전 커밋 타임아웃 - 다음 라운드로 진행
cs.enterNewRound(ti.Height, ti.Round+1)
}
}
```

2. 라운드 진행을 통한 복구

```
func (cs *State) enterNewRound(height  int64, round  int32) {
// 라운드 증가로 인한 제안자 변경
cs.Validators.IncrementProposerPriority(1)
// 새로운 제안자 선택
proposer := cs.Validators.GetProposer()
cs.Logger.Info("Entering new round",
```

```
"height", height,
"round", round,
"proposer", proposer.Address,
)
// 상태 초기화
cs.updateRoundStep(round, cstypes.RoundStepNewRound)
cs.Proposal = nil
cs.ProposalBlock = nil
cs.ProposalBlockParts = nil
// 제안 단계로 진행
cs.enterPropose(height, round)
}
```

3. 네트워크 분할 복구

```
func (cs *State) catchupReplay(height int64) error {
// WAL에서 누락된 메시지 재생
gr, found, err := cs.wal.SearchForEndHeight(height, &WALSearchOptions{
IgnoreDataCorruptionErrors: true,
})
if err != nil {
return err
}
```

```go
if !found {
return fmt.Errorf("WAL does not contain height %d", height)
}
defer gr.Close()
// 메시지 재생
dec := NewWALDecoder(gr)
for {
msg, err := dec.Decode()
if err == io.EOF {
break
}
if err != nil {
return err
}
// 메시지 처리
cs.handleMsg(msg)
}
return nil
}
```

증거 만료 및 정리

1. 증거 만료 조건

```go
func (evpool *Pool) isExpired(height int64, time time.Time) bool {
params := evpool.State().ConsensusParams.Evidence
// 블록 높이 기준 만료
ageNumBlocks := evpool.State().LastBlockHeight - height
heightExpired := ageNumBlocks > params.MaxAgeNumBlocks
// 시간 기준 만료
ageDuration := evpool.State().LastBlockTime.Sub(time)
timeExpired := ageDuration > params.MaxAgeDuration
// 둘 다 만족해야 만료
return heightExpired && timeExpired
}
```

2. 만료된 증거 정리

```go
func (evpool *Pool) removeExpiredPendingEvidence() (int64, time.Time) {
iter, err := dbm.IteratePrefix(evpool.evidenceStore, evpool.dbKeyLayout.PrefixTo
if err != nil {
return evpool.State().LastBlockHeight, evpool.State().LastBlockTime
}
defer iter.Close()
blockEvidenceMap := make(map[string]struct{})
for ; iter.Valid(); iter.Next() {
ev, err := bytesToEv(iter.Value())
```

```
if  err != nil {
continue
}
if !evpool.isExpired(ev.Height(), ev.Time()) {
// 만료되지 않은 첫 번째 증거 발견
// 다음 정리 시점 계산
nextPruningHeight := ev.Height() + evpool.State().ConsensusParams.Evidence.MaxAg
nextPruningTime := ev.Time().Add(evpool.State().ConsensusParams.Evidence.MaxAgeD
if  len(blockEvidenceMap) != 0 {
evpool.removeEvidenceFromList(blockEvidenceMap)
}
return  nextPruningHeight, nextPruningTime
}
// 만료된 증거 제거
evpool.removePendingEvidence(ev)
blockEvidenceMap[evMapKey(ev)] = struct{}{}
}
// 모든 증거가 만료되었거나 증거가 없음
if  len(blockEvidenceMap) != 0 {
evpool.removeEvidenceFromList(blockEvidenceMap)
}
return  evpool.State().LastBlockHeight, evpool.State().LastBlockTime
```

```
}
```

성능 및 보안 고려사항

1. 증거 크기 제한

```
// 블록당 최대 증거 크기 제한
func (evpool *Pool) PendingEvidence(maxBytes int64) ([]types.Evidence, int64) {
if evpool.Size() == 0 {
return []types.Evidence{}, 0
}
evidence, size, err := evpool.listEvidence(
evpool.dbKeyLayout.PrefixToBytesPending(),
maxBytes,
)
if err != nil {
evpool.logger.Error("Unable to retrieve pending evidence", "err", err)
return []types.Evidence{}, 0
}
return evidence, size
}
```

2. 메트릭 수집

```go
// 비잔틴 행위 메트릭
func (cs *State) recordByzantineMetrics(block *types.Block) {
var (
byzantineValidatorsPower = int64(0)
byzantineValidatorsCount = int64(0)
)
for _, ev := range block.Evidence.Evidence {
if dve, ok := ev.(*types.DuplicateVoteEvidence); ok {
if _, val := cs.Validators.GetByAddressMut(dve.VoteA.ValidatorAddress); val !=
byzantineValidatorsCount++
byzantineValidatorsPower += val.VotingPower
}
}
}
cs.metrics.ByzantineValidators.Set(float64(byzantineValidatorsCount))
cs.metrics.ByzantineValidatorsPower.Set(float64(byzantineValidatorsPower))
}
```

3. DoS 공격 방지

```go
// 증거 풀 크기 제한
const MaxPendingEvidenceSize = 1000
```

```go
func (evpool *Pool) AddEvidence(ev types.Evidence) error {
// 크기 제한 확인
if evpool.Size() >= MaxPendingEvidenceSize {
return fmt.Errorf("evidence pool is full")
}
// 중복 증거 확인
if evpool.isPending(ev) || evpool.isCommitted(ev) {
return nil // 조용히 무시
}
// 증거 검증 및 추가
if err := evpool.verify(ev); err != nil {
return types.NewErrInvalidEvidence(ev, err)
}
return evpool.addPendingEvidence(ev)
}
```

디버깅 및 모니터링

1. 로깅

```go
// 상세한 비잔틴 행위 로깅
func (cs *State) logByzantineBehavior(evidence types.Evidence) {
```

```go
switch  ev := evidence.(type) {
case *types.DuplicateVoteEvidence:
cs.Logger.Warn("Detected duplicate vote evidence",
"validator", ev.VoteA.ValidatorAddress,
"height", ev.Height(),
"round", ev.VoteA.Round,
"vote_type", ev.VoteA.Type,
"block_a", ev.VoteA.BlockID,
"block_b", ev.VoteB.BlockID,
"validator_power", ev.ValidatorPower,
"total_power", ev.TotalVotingPower,
)
case *types.LightClientAttackEvidence:
cs.Logger.Warn("Detected light client attack evidence",
"common_height", ev.CommonHeight,
"conflicting_height", ev.ConflictingBlock.Height,
"byzantine_validators", len(ev.ByzantineValidators),
"total_power", ev.TotalVotingPower,
)
}
}
```

2. 알림 시스템

```go
// 심각한 비잔틴 행위에 대한 알림
func (cs *State) checkByzantineThreshold(evidence types.Evidence) {
switch  ev := evidence.(type) {
case *types.DuplicateVoteEvidence:
// 전체 투표권의 10% 이상이 비잔틴 행위를 하는 경우 경고
byzantineRatio := float64(ev.ValidatorPower) / float64(ev.TotalVotingPower)
if  byzantineRatio > 0.1 {
cs.Logger.Error("High byzantine validator power detected",
"ratio", byzantineRatio,
"validator", ev.VoteA.ValidatorAddress,
)
// 외부 모니터링 시스템에 알림
cs.eventBus.PublishEventByzantineValidator(types.EventDataByzantineValidator{
Evidence: ev,
Ratio: byzantineRatio,
})
}
}
}
```

참고 자료

- [비잔틴 장애 허용성 논문]

- CometBFT 증거 처리 ADR
- Tendermint 보안 모델

이전 문서: 02-합의-단계별-분석.md

다음 문서: 04-성능-최적화.md